HARNESS LAB · MÓDULO 1 · AULA 1.2

O que acontece quando você *aperta Enter*

Você digita uma frase. O modelo recebe muito mais que isso, e não guarda nada. Cada Enter vira uma sequência de chamadas de API, e tudo o que entra nelas disputa a mesma atenção.

`chamada de API` `loop do agente` `contexto` `prompt caching` `tokens` `atenção` `/context` `/clear` `/rewind` `status line`

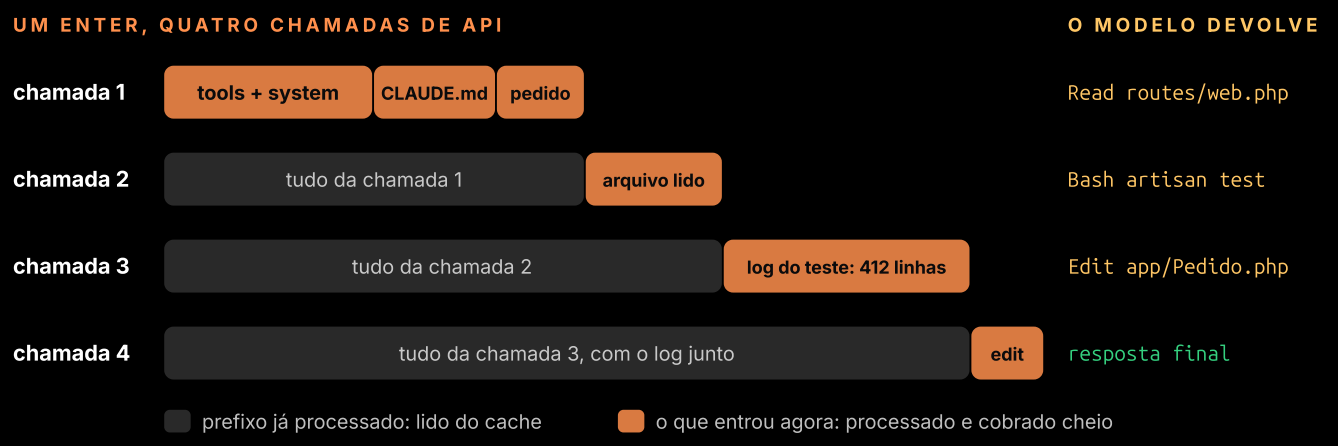
VOCÊ SAI COM

A tabela de contexto do seu projeto, antes e depois de uma tarefa curta, e uma status line que mostra o tempo todo quanto da janela já foi, mudando de cor nos 30% da aula.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

Com modelos cada vez mais inteligentes, por que ainda construir harness? Porque o modelo sozinho não faz milagre. O Claude Code (como o Codex e o OpenCode) é o harness no seu computador: dá ao agente as ferramentas de achar arquivo, escrever, apagar e rodar comando. Por trás, cada Enter vira chamadas de API ao modelo do provider, num ping-pong: o modelo olha o catálogo de ferramentas e pede "leia isto" ou "rode aquilo"; o Claude Code executa e manda o resultado numa nova chamada, até vir a resposta. O modelo lê, pede e pensa. O Claude Code executa, guarda o histórico, gerencia o contexto e monta a próxima chamada.



O que vai em cada chamada é o contexto: as ferramentas (nome, descrição e parâmetros, inclusive skills, subagentes e MCP), o system prompt, as suas mensagens e cada uso de ferramenta com a resposta. Tudo soma tokens. O modelo não reprocessa o que já viu: os providers trabalham com prefixo estável, e o começo repetido sai do cache, mais barato. Mas todo o contexto, cacheado ou não, compete por atenção.

Daí a teoria do primeiro erro, tirada de agentes de venda e de cobrança em produção: quando o modelo erra a primeira vez, as respostas seguintes degradam. Gerou código que não atende a regra de negócio? `/clear` e comece de novo, para o erro sair do contexto.

Atenção é o mecanismo do artigo *Attention Is All You Need* (Google, 2017). O modelo não lê palavras: o texto vira tokens, cada token é um ID que aponta para um vetor de significado (embedding), e cada token é comparado com todos os anteriores. Os pesos somam sempre 100%. No exemplo da aula, a regra "nunca rode `migrate:fresh`" do CLAUDE.md começa com 24% e, depois de logs, arquivos e migrations, cai para 7%. O banco está inconsistente, e ele roda. Não é burrice: é diluição de atenção.

Na demonstração, uma sessão recém-aberta já ocupava 33 mil tokens de 1 milhão (3%) no `/context`, sem nenhum pedido. Uma busca na web levou a sessão de cerca de 30 mil para 90 mil tokens, quase tudo em Messages. A dica da aula: trabalhe com no máximo 30% da janela. E como cada token se compara com cada token, a conta é quadrática, o assunto da próxima aula.

Um Enter, várias chamadas

O modelo pede, o Claude Code executa, o resultado volta. Uma frase sua vira um grande ping-pong de chamadas de API.

O modelo pensa, o harness mexe

O modelo não toca no seu disco. Quem lê, edita e roda é o Claude Code, o harness.

Cache é desconto, não esquecimento

O prefixo não é reprocessado, mas continua na janela e continua disputando atenção.

Errou cedo, recomece

O primeiro erro fica no contexto e puxa as respostas seguintes. Tirar ele de lá vale mais que remendar.

> *"The model doesn't remember anything between requests, so Claude Code re-sends the full context: the system prompt, your project context, every prior message and tool result, and your new message."*
>
> Documentação oficial do Claude Code · How Claude Code uses prompt caching

2 Indo *além da aula*

ALÉM DA AULA · O CACHE EXPIRA, E ALGUMAS AÇÕES JOGAM ELE FORA

O prefixo cacheado vive pouco: cinco minutos sem atividade com API key ou créditos de uso, uma hora na assinatura dentro do plano. Voltou do café depois disso? A primeira chamada relê a conversa inteira sem cache: mais lenta e mais cara. O mesmo vale no meio da sessão quando você troca de modelo (cada modelo tem o próprio cache), liga o fast mode ou roda `/compact`: escolha modelo e effort no começo. E editar o CLAUDE.md no meio da sessão não muda a sessão. A versão nova só entra depois de `/clear`, `/compact` ou reiniciar. O `/usage` mostra a linha `Prompt cache (main)` com a fração do input que veio do cache.

Fonte: How Claude Code uses prompt caching, "Cache lifetime", "Actions that invalidate the cache" e "Editing CLAUDE.md mid-session"; Manage costs, "Prompt cache statistics".

ALÉM DA AULA · O PRIMEIRO ERRO TEM UM ATALHO ALÉM DO /CLEAR

O `/clear` zera a conversa, mas não desfaz o código que o erro deixou. O `/rewind` (ou `Esc` duas vezes com o prompt vazio) volta para antes do pedido que gerou o erro, com "Restore code and conversation": desfaz as edições e tira o erro do contexto, mantendo o que estava bom antes. E ainda reaproveita o cache daquele ponto. Atenção: arquivo que só foi mudado por comando no terminal não é rastreado; para esse, use o git.

O estudo mais perto da teoria do primeiro erro: em mais de 200 mil conversas simuladas, o mesmo pedido entregue em vários turnos rendeu **39% menos** em média que numa mensagem só (versão mais recente do paper). O modelo assume algo cedo e não se recupera, já com 2 turnos. A recomendação dos autores: abrir conversa nova, com o pedido completo numa mensagem só.

Fontes: Checkpointing, "Rewind and summarize" e "Bash command changes not tracked"; Laban et al., *LLMs Get Lost In Multi-Turn Conversation*, ICLR 2026.

ALÉM DA AULA · OS 60 MIL TOKENS DA BUSCA: MANDE UM SUBAGENTE

Resultado de ferramenta, em regra, fica na pilha até a compactação e viaja em toda chamada seguinte. A busca na web da aula, um log de teste longo, um `cat` de arquivo grande. O subagente tem janela de contexto própria: ele lê, pesquisa e roda, e só o resumo volta para a sua conversa. Peça assim:

Dentro do Claude Code — sessão

```
> Use um subagente para pesquisar isso e me traga um resumo de 10 linhas
```

Fonte: Manage costs, "Delegate verbose operations to subagents"; Explore the context window. Quando o subagente vale a pena (e quando não) é a aula 3.4.

ALÉM DA AULA · O QUE O /CONTEXT SOMA E O QUE ELE SÓ MOSTRA

Linhas com "(deferred)" aparecem na tabela, mas ficam fora do total. É o tool search: do MCP, só os nomes das ferramentas e as instruções do servidor entram na sessão, e a definição completa carrega quando o modelo precisa. Teste feito para este material, numa máquina com muitos MCPs: `MCP tools (deferred)` marcava 427,8 mil tokens, e o total da sessão era 23,6 mil. Das skills, também só a descrição entra no começo. Já o CLAUDE.md da raiz entra inteiro, em toda chamada. O `Autocompact buffer` é a reserva para a compactação automática.

Fontes: Connect Claude Code to tools via MCP, "Scale with MCP tool search"; Explore the context window, "Before you type anything".

ALÉM DA AULA · 30% É REGRA DE PRÁTICA, E A JANELA MUDA DE TAMANHO

Os 30% são a prática do Lucas, não um número de estudo. O que os estudos mostram vai na mesma direção: só o aumento da entrada, dentro da janela anunciada, derrubou a acurácia de **13,9% a 85%**, mesmo com o modelo recuperando 100% da informação relevante, e até com o texto extra trocado por espaço em branco. E 30% de quê? A janela é de 200 mil ou de 1 milhão, conforme o modelo e o plano: 30% de 1M são 300 mil tokens, mais do que uma janela inteira de 200 mil. O `/context` e a status line desta aula mostram as duas coisas: o tamanho da janela e quanto dela já foi.

Fonte: Du, Tian, Peng et al., *Context Length Alone Hurts LLM Performance Despite Perfect Retrieval*, arXiv 2510.05381; Explore the context window, "When your context fills up".

ALÉM DA AULA · 24% PARA 7% É ILUSTRAÇÃO, E O ORÇAMENTO É REAL

As porcentagens do `migrate:fresh` são didáticas: o mecanismo real roda em várias cabeças em paralelo e em dezenas de camadas. O que não é ilustração é o orçamento. A Anthropic: "Every new token introduced depletes this budget by some amount." A regra não some do contexto; fica lá, inteira, com uma fatia menor.

Fonte: Anthropic, *Effective context engineering for AI agents*, 2025.

3 Faça *no seu repositório*

A prática é medir quanto do contexto já está ocupado antes da sua primeira palavra, e quanto uma tarefa curta acrescenta. Para o número ficar à vista, a aula instala uma status line: o percentual da janela embaixo do prompt, verde até 30%, amarelo até 60%, vermelho acima. Vai para o seu repositório: `.claude/statusline-contexto.sh` e a chave `statusLine` no `.claude/settings.json`. Nada seu é apagado.

1 Confira o jq.

A status line lê o JSON do Claude Code com ele. Tem que responder com a versão:

Terminal · qualquer pasta — shell

```
jq --version
```

Não tem? Instale com o gerenciador do seu sistema:

Ubuntu e WSL — shell

```
sudo apt install -y jq
```

macOS — shell

```
brew install jq
```

2 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

3 Instale os arquivos da aula, na raiz do seu projeto.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 1.2
```

A saída mostra `criado` ou `somado` para cada arquivo. Se o seu projeto já tem uma `statusLine`, a sua fica e a do curso não liga. Para usar a do curso, troque o `command` da sua por `"$CLAUDE_PROJECT_DIR"/.claude/statusline-contexto.sh`.

4 Teste a status line fora do Claude Code.

Ela é só um programa: entra JSON no stdin, sai uma linha. Mande o que o Claude Code mandaria com 35% da janela usada:

Terminal · na raiz do seu projeto — shell

```
j='{"context_window":{"used_percentage":35}}'
echo "$j" | .claude/statusline-contexto.sh
```

o que aparece — em amarelo no terminal

```
contexto 35% · passou de 30%: feche a tarefa e use /clear
```

5 **Tire a foto do custo fixo, antes de abrir a sessão.**

Em modo não interativo, o `/context` sai em markdown, sem chamar o modelo. Ele mostra o estado de uma sessão nova: é a coluna "Antes" da tabela.

Terminal · na raiz do seu projeto — claude

```
claude -p "/context" < /dev/null > ~/contexto-antes.md
head -22 ~/contexto-antes.md
```

A foto fica fora do projeto, para não ir parar num commit. O `< /dev/null` evita a espera de 3 segundos por um stdin que não vem.

6 **Abra uma sessão nova e rode /context antes de digitar.**

Na raiz do projeto, rode `claude` sem `--continue` nem `--resume` e aceite a confiança na pasta (sem isso, a status line fica em branco). Antes de qualquer pedido, a linha embaixo do prompt diz que o número aparece depois da primeira resposta. O `/context` já mostra:

Dentro do Claude Code — sessão

```
/context
```

Anote o total e quanto vem de System prompt, System tools, MCP, Memory files (o seu CLAUDE.md) e Skills. Esse é o custo fixo: vai em toda chamada, antes da sua primeira palavra.

7 **Peça uma tarefa curta, real e só de leitura.**

Enquanto ele trabalha, conte as ferramentas que ele chama: cada leitura ou comando é mais uma chamada de API. Troque o arquivo pelo da sua stack: `config/routes.rb` no Rails, o arquivo onde o router é montado no Node, `urls.py` no Django.

Dentro do Claude Code — sessão

```
> Leia routes/web.php e me diga em 5 linhas como chega ao banco
```

8 **Rode /context de novo e compare.**

Dentro do Claude Code — sessão

```
/context
/usage
```

A linha que mais muda é Messages: é a conversa, que vai inteira na próxima chamada. A status line agora mostra o percentual e o cache. O `/usage` mostra quanto do input saiu do cache.

9 **Treine a saída do primeiro erro.**

Da próxima vez que o modelo errar a regra de negócio, em vez de corrigir por cima: `/rewind`, escolha o pedido que gerou o erro (o Claude Code volta para antes dele e devolve o texto ao input) e reescreva o pedido completo. Se a tarefa acabou, ou o assunto é outro, `/clear`.

Dentro do Claude Code — sessão

```
/rewind
/clear
```

A sua tabela de contexto

Também em `modelos/tabela-contexto.md`, com pedidos prontos para cada stack.

CATEGORIA NO /CONTEXT	ANTES (SESSÃO NOVA)	DEPOIS (TAREFA CURTA)
Total usado (tokens e %)		
System prompt		
System tools		
MCP tools e MCP server instructions		
Memory files (CLAUDE.md, auto memory)		
Skills e Custom agents		
Messages		
Free space		
Ferramentas chamadas na tarefa	não se aplica	

Feche com uma frase: "a categoria que mais pesa no meu custo fixo é ____, e ela precisa (ou não) estar em toda chamada". Não corte nada ainda: o CLAUDE.md enxuto é a aula 3.1, e o que entra e o que fica fora da sessão é a 3.5.

Os arquivos que o instalador colocou

`.claude/settings.json` — instalado pelo instalar.sh · não copie do PDF

```
{
  "statusLine": {
    "type": "command",
    "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/statusline-contexto.sh"
```

```
  }
}
```

O Claude Code roda o comando a cada resposta nova, depois de um `/compact` e quando o modo de permissão muda, com um JSON no stdin. O `$CLAUDE_PROJECT_DIR` aponta para a raiz onde a sessão começou (conferido numa sessão real do 2.1.316). O script lê do JSON `context_window.used_percentage` (`null` antes da 1ª resposta), `context_window_size` (200 mil ou 1 milhão) e `prompt_cache.warm`, `ttl` e `expires_at`, e escolhe a cor pelo `LIMITE`, 30 por padrão.

O percentual vem da última chamada: tokens de entrada, com o cache incluído, sobre a janela inteira. O `/context` estima por categoria, então os dois podem diferir um pouco. Para mudar o limite, ponha `HL_CONTEXTO_LIMITE=40` na frente do `command`. Já tem uma status line pessoal no `~/.claude/settings.json`? A do projeto vence a sua. O `README.md` da aula mostra como ficar com a sua ou levar esta para o nível do usuário.

4 Arquivos que *reforçam a aula*

`.claude/statusline-contexto.sh`

A regra dos 30% vira cor na tela. O contexto deixa de ser algo que você consulta de vez em quando e passa a estar à vista a cada resposta, junto com o estado do cache.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.2-o-que-acontece-quando-voce-aperta-enter/projeto/.claude/statusline-contexto.sh

`.claude/settings.json`

Liga a status line no projeto. Commitado, o time inteiro vê o mesmo número e fala a mesma língua sobre contexto.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.2-o-que-acontece-quando-voce-aperta-enter/projeto/.claude/settings.json

`modelos/tabela-contexto.md`

A prática da aula em arquivo: a tabela antes e depois, os pedidos curtos por stack e a frase de fechamento. Guarde preenchida: ela volta no Módulo 3, quando você for cortar o custo fixo.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.2-o-que-acontece-quando-voce-aperta-enter/modelos/tabela-contexto.md

`modelos/settings.statusline-no-usuario.json`

A mesma status line só para você, em todos os projetos, sem mexer no repositório do time. Copie o script para `~/.claude/` e use este bloco no seu settings de usuário.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.2-o-que-acontece-quando-voce-aperta-enter/modelos/settings.statusline-no-usuario.json

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
A status line não aparece	A pasta ainda não foi confiada, o script está sem permissão de execução ou `disableAllHooks` está ligado	Aceite o diálogo de confiança; `chmod +x .claude/statusline-contexto.sh`; `claude --debug` mostra o motivo

SINTOMA	CAUSA	CORREÇÃO
Ela diz "aparece depois da 1ª resposta"	Antes da primeira resposta da sessão, o Claude Code manda o percentual como `null`	Normal. Mande o primeiro pedido, ou rode `/context`
No passo 8, ficou a status line antiga do projeto	O projeto já tinha `statusLine`, e o instalador mantém a sua	Troque o `command` dela por `"$CLAUDE_PROJECT_DIR"/.claude/statusline-contexto.sh`
A sua status line pessoal sumiu	A do projeto vence a do `~/.claude/settings.json`	Apague a chave `statusLine` do settings do projeto, ou leve esta para o usuário (README da aula)
`claude -p "/context"` fica 3 segundos parado e avisa "no stdin data received"	Sem stdin redirecionado, ele espera entrada	Use `< /dev/null`, como no passo 5
Os números não batem com os do vídeo	O custo fixo depende de MCPs, plugins, skills, modelo e versão	Compare você com você: antes e depois, no mesmo projeto
Editou o CLAUDE.md e o agente seguiu a versão velha	O CLAUDE.md é lido no começo da sessão e fica no prefixo cacheado até o próximo `/clear`, `/compact` ou reinício	`/clear` ou reinicie a sessão
Depois de uma pausa, a primeira resposta demora e custa mais	O cache expirou (5 min com API key, 1 h na assinatura)	Normal. Na volta, se a tarefa mudou, prefira `/clear` a continuar a conversa velha

6 Você sai *com*

- [ ] A tabela de contexto do seu projeto preenchida, antes e depois de uma tarefa curta.
- [ ] O seu custo fixo em tokens, em `~/contexto-antes.md`.
- [ ] O número de ferramentas que uma tarefa curta gerou.
- [ ] A status line ligada: `contexto N%` embaixo do prompt, verde até 30%.
- [ ] Uma frase dizendo qual categoria mais pesa e se ela precisa estar em toda chamada.

PROMETIDO NA AULA

- ✓ "Abre aí o seu Claude Code (...) e dá o comando /context": passos 5 a 8 da seção 3, com a tabela.
- ✓ "Sempre trabalhar com no máximo 30% da sua janela de token": a status line da seção 3 muda de cor nos 30%.
- ✓ "A melhor coisa é /clear e começar de novo": passo 9 da seção 3, e o `/rewind` na seção 2.
- ✓ "Um problema muito clássico, que a gente vai ver na próxima aula": a Próxima aula, no fim.

7 *Glossário*

Chamada de API	Uma ida e volta entre o Claude Code e o modelo, com tudo o que ele vê.
Loop do agente	O modelo pede uma ferramenta, o harness executa, o resultado volta para o contexto, nova chamada. Até vir a resposta.
Harness	A camada em volta do modelo: ferramentas e contexto. O Claude Code é um.
Contexto	Tudo o que vai numa chamada: ferramentas, system prompt, CLAUDE.md, histórico e a mensagem nova. A única memória do modelo.
Janela de contexto	O máximo de tokens que cabe numa chamada: 200 mil ou 1 milhão, conforme o modelo. Caber não quer dizer render igual.
Embedding	O vetor que representa o significado de um token. O modelo manipula isso.
Atenção	Compara cada token com os anteriores; os pesos somam 100%. Orçamento finito.
Prompt caching	O provider guarda o prefixo repetido e cobra menos para reler. Expira parado.
Tool search	Ferramentas de MCP entram só pelo nome; a definição completa carrega quando o modelo precisa. As linhas "(deferred)" do `/context`.

8 Para ir *além*

- How Claude Code uses prompt caching: o que fica no cache, quanto dura e o que joga ele fora.
- Explore the context window: simulação de uma sessão enchendo a janela, do que carrega antes do primeiro pedido até a compactação.
- Effective context engineering for AI agents (Anthropic, 2025): o "orçamento de atenção" e o n^2.
- LLMs Get Lost In Multi-Turn Conversation (ICLR 2026): os 39% e a conversa nova quando travar.
- Context Length Alone Hurts LLM Performance Despite Perfect Retrieval (2025): o tamanho sozinho derruba o resultado, até com espaço em branco no lugar do texto extra.

Próxima: *Aula 1.3 · Por que o agente esquece a regra*

A conta quadrática da atenção cobra o preço: o agente sabe a regra, recita a regra se você perguntar, e quebra a regra mesmo assim.